

Object-Oriented Programming in Java

Lecture 9 - Input and Output

Emily Lucia Antosch

HAW Hamburg

03.10.2025

Inhaltsverzeichnis

1. Introduction	2
2. Stream concept & Screen output	6
3. Keyboard input	12
4. Byte & Character Streams	25
5. Files	32
6. License Notice	38

1. Introduction

1.1 Where are we now?

- In the last lecture, we dealt with handling exceptions
- You can now
 - ▶ throw and catch exceptions,
 - ▶ handle exceptions with `try` and `catch`
 - ▶ and define your own exception types.
- Today we continue with **Input and Output**.

1.1 Where are we now?

1. Introduction

1. Imperative Concepts
2. Classes and Objects
3. Class Libraries
4. Inheritance
5. Interfaces
6. Graphical User Interfaces
7. Exception Handling
8. **Input and Output**
9. Multithreading (Parallel Computing)

1.2 The goal of this chapter

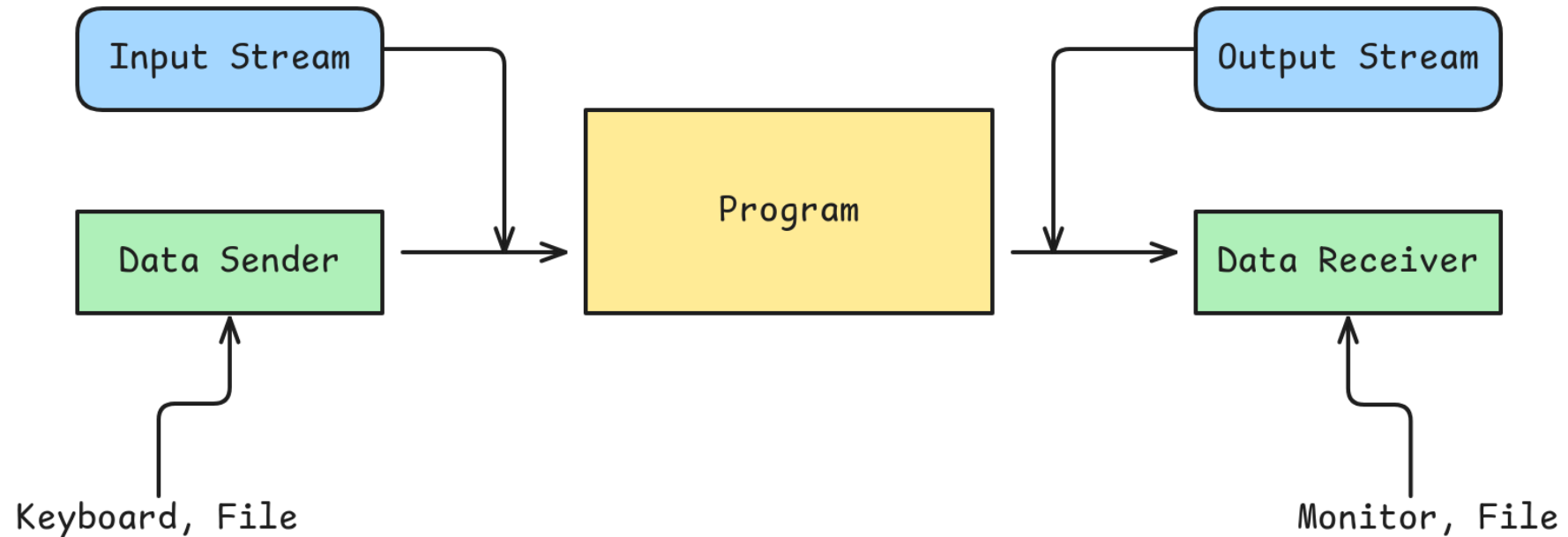
- You read characters, strings and numeric values from the keyboard.
- You chain and use input and output streams contained in the Java SDK for input and output of bytes, characters and text lines.
- You read and write strings from and to text files.

2. Stream concept & Screen output

2.1 Stream concept

2. Stream concept & Screen output

- Stream: Transports data from sender („source“) to receiver („sink“)
- Input: Reading data into a program
- Output: Data leaves a program
- Class library contains about 50 classes for all important input and output variants



2.1 Stream concept

2. Stream concept & Screen output

- With what we have already learned:
 - ▶ What are the components of `System.out.println()`?
- Only this makes sense:
 - ▶ `System`: Class (since no variable `System` is declared)
 - ▶ `out`: Class variable of `System`, references an object
 - ▶ `println()`: Method of the object referenced via `out`
- Output stream:
 - ▶ `System.out` references object of class `PrintStream`
 - ▶ Object is connected to the screen

2.1 Stream concept

2. Stream concept & Screen output

- Selected methods of the `PrintStream` class:

Methods	Meaning
<code>println(String message)</code>	Output with line break(print line)
<code>print(String message)</code>	Output without line break
<code>printf(String format, Object... arg)</code>	Formatted Output (see <code>String.format()</code>)
<code>format(String format, Object... arg)</code>	Formatted Output (see <code>String.format()</code>)

Tabelle 1: Formats and Flags

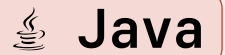
2.1 Stream concept

2. Stream concept & Screen output

? Frage

- What is output?

```
1    public static void main(String[] args) {  
2        double tempHawaiiCelsius = 15.97;  
3        double tempHamburgCelsius = 22.71;  
4        String.format("Hawaii: %.1f °C", tempHawaiiCelsius);  
        System.out.printf("Hamburg: %.1f °C", tempHamburgCelsius);  
5    }
```



2.1 Stream concept

2. Stream concept & Screen output

Streams referenced in System:

Reference	Data Type	Meaning
System.out	PrintStream	Output on screen
System.err	PrintStream	Error output on screen
System.in	InputStream	Input from keyboard

Tabelle 2: Formats and Flags

3. Keyboard input

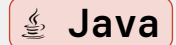
3.1 Class Scanner

- Provides methods for reading texts and simple data types (e.g. int)
- Text input is analyzed and interpreted („parsing“, e.g. converting to integer)
- Creation and termination:
 - ▶ Scanner object is connected to input stream in constructor
 - ▶ The connection should be terminated via the Scanner method `close()`.



Beispiel

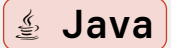
```
1 public class ScannerLine {
2     public static void main(String[] args) {
3         Scanner scanner = new Scanner(System.in);
4
5         System.out.print("Bitte einen Satz eingeben: ");
6         System.out.println(scanner.nextLine());
7         scanner.close();
8     }
9 }
```



? Frage

- Oops, what happens here?

```
1 public class ScannerToken {
2     public static void main(String[] args) {
3         Scanner scanner = new Scanner(System.in);
4
5         System.out.print("Please enter a sentence: ");
6         System.out.println(scanner.next());
7         scanner.close();
8     }
9 }
```



- Method `next()`: Only first word instead of entire sentence is read and output
- Words and lines are distinguished.

3.1 Class Scanner

- Separators of multiple inputs:
 - ▶ Token: Individual words or values (e.g. integer)
 - ▶ Tokens in input separated by separators
 - ▶ Default separator is a whitespace (i.e. space, tab, line break)
- Methods:
 - ▶ Separator changeable via method `useDelimiter()`
 - ▶ Via method `hasNext()` query whether tokens are still available

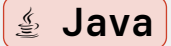
☰ Aufgabe 1

- Write a program that reads a sentence via `next()`.

3.1 Class Scanner

3. Keyboard input

```
1  public class ScannerNext {
2      public static void main(String[] args) {
3          Scanner scanner = new Scanner(System.in);
4
5          System.out.print("Bitte einen Satz eingeben: ");
6          while (scanner.hasNext()) {
7              System.out.println(scanner.next());
8          }
9          scanner.close();
10     }
11 }
```



? Frage

- What happens if you replace `scanner.hasNext()` with `true`?
- How does `next()` behave once all words have been read?

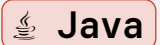
3.1 Class Scanner

- Spezielle Methoden für einfache Datentypen:
 - ▶ Einlesen: `nextBoolean()`, `nextInt()`, `nextDouble()`, ...
 - ▶ Abfrage: `hasNextBoolean()`, `hasNextInt()`, `hasNextDouble()`, ...

? Frage

- Welche Ausgaben werden für die Eingaben „127“, „128“ und „Hamburg“ erzeugt?

```
1 public class ScannerByte1 {
2     public static void main(String[] args) {
3         Scanner scanner = new Scanner(System.in);
4
5         System.out.print("Please enter a byte value: ");
6         System.out.println("Entered: " + scanner.nextByte());
7         scanner.close();
8     }
9 }
```



3.1 Class Scanner

- Parsing errors:
 - ▶ Inputs „128“ and „Hamburg“: Exception of type `InputMismatchException`
 - ▶ Has base class `RuntimeException` (exception handling not mandatory)

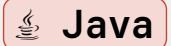
☰ Aufgabe 2

- The program should not be terminated by an exception:
 - Find two different ways to avoid this.
 - Implement these approaches.
- Approaches:
 - ▶ Catch the exception
 - ▶ Query via `hasNextByte()`

3.1 Class Scanner

- Catch exception:

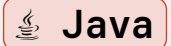
```
1  public class ScannerByte2 {
2      public static void main(String[] args) {
3          Scanner scanner = new Scanner(System.in);
4
5          System.out.print("Please enter a byte value: ");
6          try {
7              System.out.println("Entered: " + scanner.nextByte());
8          } catch (InputMismatchException e) {
9              System.out.println("Input is not a byte value.");
10         } finally {
11             scanner.close();
12         }
13     }
14 }
```



3.1 Class Scanner

- Query data type:

```
1  public class ScannerByte3 {
2      public static void main(String[] args) {
3          Scanner scanner = new Scanner(System.in);
4
5          System.out.print("Please enter a byte value: ");
6          if (scanner.hasNextByte()) {
7              System.out.println("Entered: " + scanner.nextByte());
8          } else {
9              System.out.println("Not a byte value: " + scanner.next());
10         }
11         scanner.close();
12     }
13 }
```



Aufgabe 3

- Read the components of a vector (data type `int`)
- Read components until another token (e.g. a letter) is entered
- Output the vector as well as the magnitude



Beispiel

Integer components (terminate with different character): 7 4 0 15 End

$$a = [7, 4, 0, 15]^T$$

$$\|a\| = 17,03$$

3.1 Class Scanner

3. Keyboard input

```
1  public class ScannerVektor {
2      public static void main(String[] args) {
3          Scanner scanner = new Scanner(System.in);
4          ArrayList<Integer> vector = new ArrayList<Integer>();
5          System.out.print("Integer components (terminate with different character): ");
6          while (scanner.hasNextInt())
7              vector.add(scanner.nextInt());
8          scanner.close();
9          if (vector.size() > 0) {
10             System.out.print("a = [" + vector.get(0));
11             long sumOfSquares = vector.get(0) * vector.get(0);
12
13             for (int i = 1; i < vector.size(); i++) {
14                 System.out.print(", " + vector.get(i));
15                 sumOfSquares += vector.get(i) * vector.get(i);
16             }
17             System.out.println("]^T");
18             System.out.printf("||a|| = %.2f\n", Math.sqrt(sumOfSquares));
19         }
20     }
21 }
```

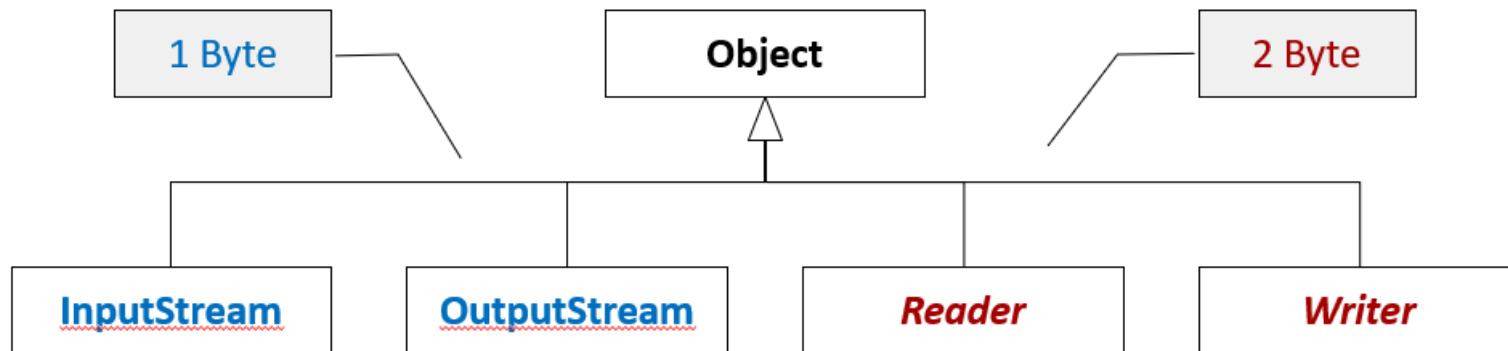


4. Byte & Character Streams

4.1 Byte & Character Streams

4. Byte & Character Streams

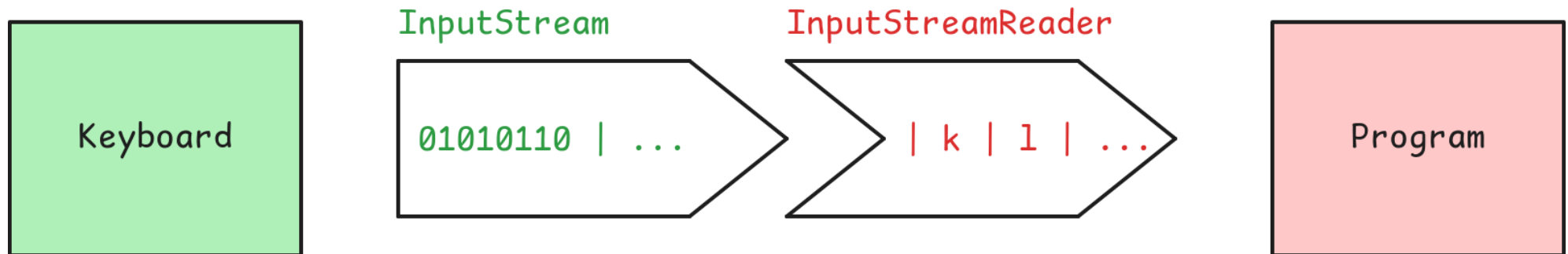
- What was the special feature of characters in Java again?
 - ▶ All characters encoded as 2 bytes (Unicode)
 - ▶ Distinguish: Streams that transport elements of 1 byte or 2 bytes („characters“)
- Byte streams (byte-oriented streams):
 - ▶ Transport individual bytes
 - ▶ Classes `InputStream` and `OutputStream` as well as classes derived from them
- Character streams (character-oriented streams):
 - ▶ Transport characters of 2 bytes each
 - ▶ Abstract classes `Reader` and `Writer` as well as classes derived from them



4.1 Byte & Character Streams

4. Byte & Character Streams

- Keyboard delivers stream of individual bytes (e.g. `System.in` of data type `InputStream`)
 - ▶ Java characters consist of 2 bytes
 - ▶ Connect byte stream with character stream
- Notes:
 - ▶ Goal in the following: Illustration of stream chaining
 - ▶ Yes, keyboard inputs (code $l = 255$) you wouldn't need to chain with a character stream.
 - ▶ Yes, feel free to use `Scanner` for keyboard inputs.



4.1 Byte & Character Streams

4. Byte & Character Streams

```
1  public class KeyboardReader1 {
2      public static void main(String[] args) throws IOException {
3          InputStreamReader reader = new InputStreamReader(System.in);
4
5          System.out.print("Please enter a character: ");
6          System.out.println(reader.read());
7          System.out.println(reader.read());
8          System.out.println(reader.read());
9          reader.close();
10     }
11 }
```



? Frage

- Why is `read()` called three times?
- Why are the second and third outputs always 13 and 10?

4.1 Byte & Character Streams

4. Byte & Character Streams

- `BufferedReader` reads a character stream and buffers the characters
- Provides e.g. method `readLine()` for reading out a line
- Analogously, class `BufferedWriter` outputs entire line via `newLine()`



☰ Aufgabe 4

- ▶ Modify the previous example as follows:
 - Read two lines via
 - `BufferedReader` Then output both lines

4.1 Byte & Character Streams

4. Byte & Character Streams

```
1  public class KeyboardReader2 {
2      public static void main(String[] args) throws IOException {
3          InputStreamReader reader = new InputStreamReader(System.in);
4          BufferedReader bufferedReader = new BufferedReader(reader);
5
6          System.out.print("Please enter first line: ");
7          String line1 = bufferedReader.readLine();
8          System.out.print("Please enter second line: ");
9          String line2 = bufferedReader.readLine();
10
11         System.out.println(line1);
12         System.out.println(line2);
13         reader.close();
14     }
15 }
```



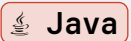
Java

5. Files

5.1 Files and Directories

- Class `File` represents file or directory
 - ▶ Objects contain information about file, not its content
 - ▶ IntelliJ uses the project directory as root directory for reading/writing.

```
1  public class CreateFile {
2      public static void main(String[] args) throws IOException {
3          File file = new File("Testdatei.txt");
4          boolean isExists = file.exists();
5
6          if (!isExists) {
7              System.out.println("Create file");
8              isExists = file.createNewFile();
9          }
10
11         if (isExists && file.isFile()) {
12             System.out.println("Read: " + file.canRead());
13             System.out.println("Write: " + file.canWrite());
14             file.delete();
15         }
16     }
17 }
```



```
1  public class ListDirectory {
2      public static void main(String[] args) {
3          File directory = new File(".");
4
5          if (directory.isDirectory()) {
6              String[] children = directory.list();
7              for (String child : children) {
8                  System.out.println(child);
9              }
10         }
11     }
12 }
```



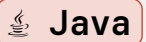
- Byte streams:
 - ▶ Read files via `FileInputStream` classes and write via `FileOutputStream`
- Character streams (e.g. text files):
 - ▶ Read files via `FileReader` and write via `FileWriter`
 - ▶ Buffered character streams via `BufferedReader` and `BufferedWriter`



5.1 Files and Directories

- Let's apply this:
 - ▶ Create a program that writes a text file.
 - ▶ Create another program that reads the content of the text file and outputs it.

```
1  public class WriteFile {
2      public static void main(String[] args) throws IOException {
3          File file = new File("Testdatei.txt");
4          FileWriter writer = new FileWriter(file);
5          BufferedWriter bufferedWriter = new BufferedWriter(writer);
6
7          bufferedWriter.write("This is the first line.");
8          bufferedWriter.newLine();
9          bufferedWriter.write("And here comes the second line.");
10         bufferedWriter.newLine();
11         bufferedWriter.close();
12     }
13 }
```



```
1  public class ReadFile {
2      public static void main(String[] args) throws IOException {
3          File file = new File("Testdatei.txt");
4          FileReader reader = new FileReader(file);
5          BufferedReader bufferedReader = new BufferedReader(reader);
6
7          while (bufferedReader.ready()) {
8              System.out.println(bufferedReader.readLine());
9          }
10         bufferedReader.close();
11     }
12 }
```



6. License Notice

6.1 Attribution

- This work is shared under the CC BY-NC-SA 4.0 License and the respective Public License
- `link(„https://creativecommons.org/licenses/by-nc-sa/4.0/“)`
- This work is based off of the work Prof. Dr. Marc Hensel.
- Some of the images and texts, as well as the layout were changed.
- The base material was supplied in private, therefore the link to the source cannot be shared with the audience.